

부록. 교차검증

머신러닝 모델 성능 검증

1 방법. 홀드 아웃

2 방법. k-겹 교차 검증

- 너무 간단한 모델 과소적합 문제, 너무 복잡한 모델 과대 적합 문제 발생
- 무작위로 분할된 데이터(훈련세트/테스트세트) 사용 모델의 경우 데이터의 편향으로 훈련 세트에 과적합 되거나 반대의 경우 발생할 수 있음

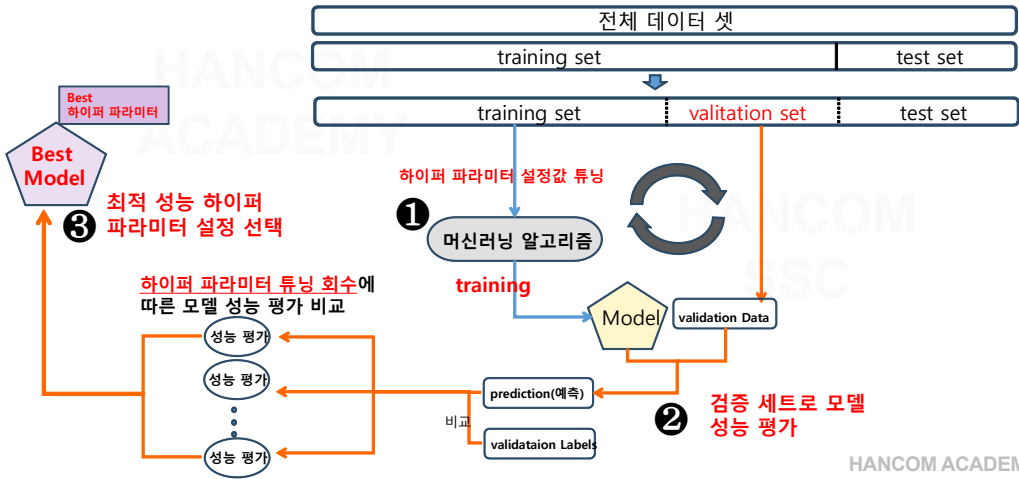
* 모델 성능 추정(검증) 목적

- 사용되지 않는 데이터에 대한 모델 예측 성능인 일반화 정확도를 추정
- 학습 알고리즘 튜닝하고 가장 성능이 뛰어난 모델 선택해서 예측 성능을 증가
- 문제에 가장 적합한 머신러닝 알고리즘 선택

* test 데이터셋은 학습 및 검증에 사용되지 않고 모델 예측 평가에만 활용

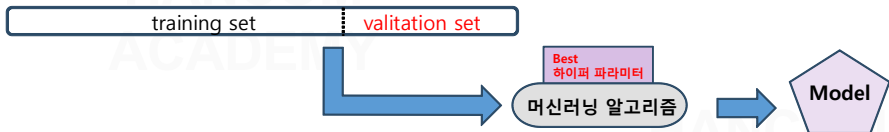
3-홀드아웃 도식화

- 훈련 / 검증 / 테스트 세트 3가지로 분할

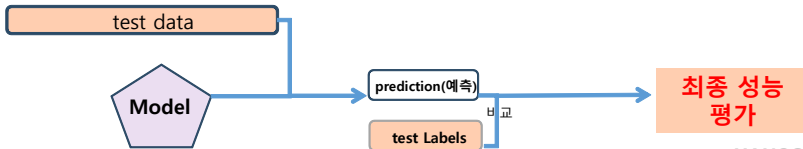


3-홀드아웃 도식화

- ④ 최적 모델만 메모리에 유지시키기 위해
전체 훈련데이터로 최적 선택 모델을 다시 훈련



- ⑤ 최종 test 데이터로 모델 일반화 정확도 추정(평가)

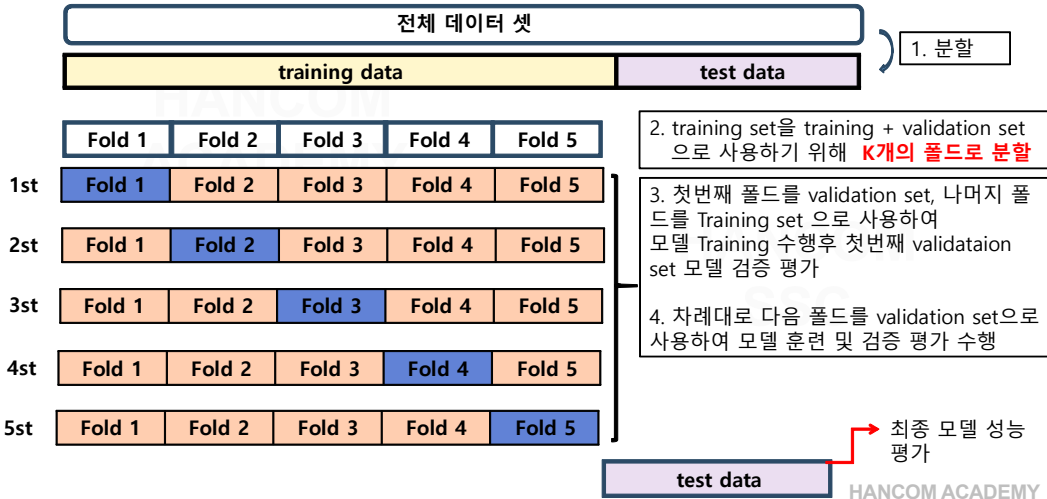


k-겹 교차 검증

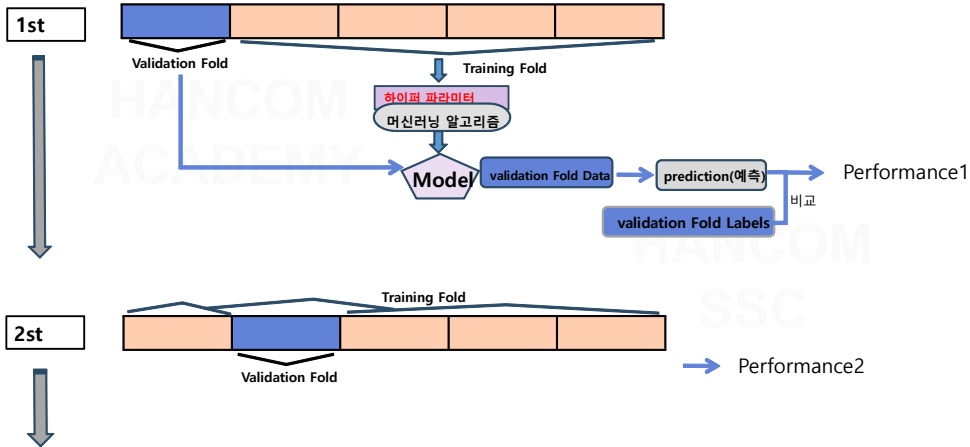
- 훈련 세트 / 검증 세트 / 테스트 세트 3개 세트로 **분할하지 않고**
훈련 세트의 일부분을 검증 세트로 활용해서 데이터의 중복을 허락하지 않고 모델 검증
- **반복적인 홀드 아웃 방법에서 검증 샘플 데이터를 재사용하는 것은 반복 과정 사이의 독립적인 성능 추정을 만들지 못함**
- 훈련데이터를 동일한 크기를 가진 K개 분할로 나눔
- **각 분할 i에 대해 남은 k-1개의 분할로 모델 훈련하고 분할 i(검증세트) 에서 모델 평가**
- **최종 검증 점수는 k개의 점수 평균**
- 데이터셋이 작은 경우 **데이터 분할에 따라 모델의 성능 편차가 클때 유용**

* 데이터 집합을 체계적으로 바꿔가면서 모든 데이터에 대해 모형의 성과를 측정하는 검증 방식

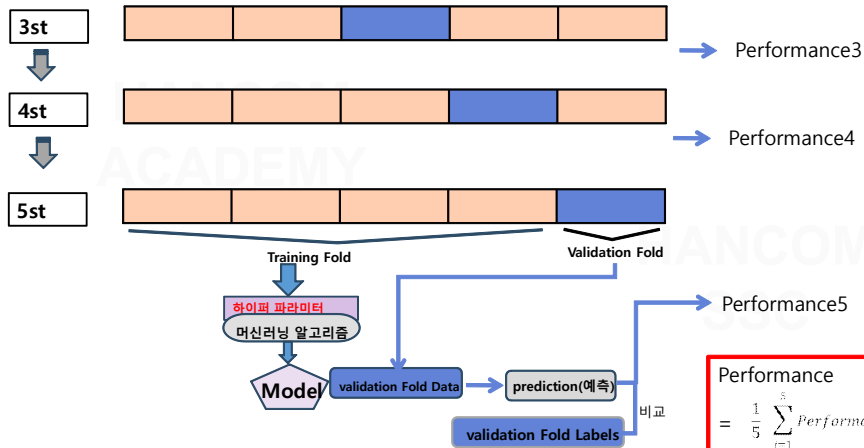
k-겹 교차 검증 도식화



5-겹 교차 검증 진행 과정



5-겹 교차 검증 진행 과정



총 k-겹 개의 성능 결과가 나오며,
이 k개 성능의 평균을 해당 학습 모델의 성능으로 간주

교차 검증 (cross_validate : 다중 평가 지표)

```
from sklearn.model_selection import cross_validate  
cross_validate(estimator, X, y=None, *, groups=None, scoring=None, cv=None,  
n_jobs=None, verbose=0, fit_params=None, pre_dispatch='2*n_jobs',  
return_train_score=False, return_estimator=False, error_score=nan)
```

estimator : 검증평가를 수행할 모델

X : 훈련 데이터 세트

Y : 타깃 데이터(레이블)

scoring : 원하는 평가 지표 지정 , 예) ['accuracy', 'r2']

cv : 교차 검증 분할 수(k), default : 5-fold cross validation (sklearn 0.20이상 버전)

n_jobs : 병렬 실행에 사용할 CPU 코어 수 지정, -1로 지정하면 시스템 모든 코어 사용

return_train_score : 명시적으로 **훈련 폴드의 점수를 받을지 여부** 설정

* 반환값

- 각 폴드에서 훈련 과 검증에 걸린 시간 (fit_time, score_time)
- scoring에 지정한 평가 지표별 훈련점수 와 검증 점수를 반환
- scoring = None 일 경우 **test_score(검증 폴드 점수) 반환**

cross_val_score() 함수 : test_score(검증 폴드 평가 점수) 만 반환

sklearn - GridsearchCV(그리드 서치)

: 하이퍼 파라미터를 순차적으로 적용하면서 최고 성능을 가지는 파라미터 조합을 찾음

- 하이퍼파라미터 튜닝과 최적 모델 선택을 위한 훈련,검증 세트 사용
- 테스트 세트는 최종 모델 평가를 위해 별도 유지
- 하이퍼파라미터 탐색과 교차 검증을 한 번에 수행 (별도 교차검증 `cross_validate()` 사용할 필요 없음)
- **decision tree** 모델 나 **svm** 모델 처럼 튜닝할 하이퍼파라미터가 많은 경우 유용

decision tree 하이퍼 파라미터 튜닝 예 경우)

```
grid_parameters = { 'max_depth':[1,2,3], 'min_sample_split':[2,3]}
```

- 하이퍼 파라미터를 (1,2),(1,3),(2,2),(2,3)...순으로 순차적으로 적용 , 총 6회 걸쳐 실행하면서 최적 파라미터와 수행 결과 도출
- 사용자가 직접 for문 활용하는것 보다 조금 더 유연한 API레벨 제공
- `cross-validation()` 기반으로 최적 파라미터 값을 찾아줌으로 훈련/검증 데이터 셋으로 자동 분할 후 파라미터를 순차적으로 적용, **편리한 대신 수행 시간이 상대적으로 오래 걸릴수 있음.**
- 만약 CV 가 3일 경우 총 $6 \times 3 = 18$ 회 교차검증 수행

sklearn - GridsearchCV(그리드 서치)

GridsearchCV 클래스 생성자의 주요 파라미터

- estimator : classifier, regressor 등의 모델
- param_grid : 튜닝할 파라미터 딕셔너리, (파라미터명과 사용될 여러 파라미터 값을 지정)
- cv : 교차 검증을 위해 분할되는 폴드 수 (default : 5)
- n_jobs : 병렬 실행에 사용할 cpu 코어 수 (default : 1) , -1 지정 시 모든 코어 사용
- refit : 가장 최적의 하이퍼 파라미터를 찾은 뒤 입력된 estimator 객체를 해당 하이퍼 파라미터로 자동 재학습 시킴(default : True)

GridsearchCV 흐름

1. 탐색할 매개변수를 지정
2. 훈련 세트에서 그리드 서치 수행하여 최상의 검증 점수가 나오는 매개변수 조합 찾음
(이 조합은 그리드 서치 객체에 저장)
3. 그리드 서치는 최상의 매개변수에서 (교차검증에 사용한 훈련세트가 아닌) 전체 훈련 세트를 사용해 최종 모델을 훈련
(이 모델도 그리드 서치 객체에 저장)

SVM 모델 GridsearchCV 사용 예

```
import pandas as pd
import numpy as np
from sklearn.datasets import load_iris
from sklearn import svm
```

```
pd.set_option('display.max_columns',100)
pd.set_option('display.width',1000)
iris = load_iris() # 붓꽃 데이터 로딩
```

```
Iris_Data = pd.DataFrame(np.column_stack([iris['data'], iris['target']]),
                        columns=['sepal_len','sepal_wd',
                                'petal_len','petal_wd','target'])
```

```
# petal_len, petal_wd == > X 데이터 추출
X_petal_len_wd = Iris_Data[['petal_len','petal_wd']]
print(X_petal_len_wd.sample(10))
```

```
# 레이블(클래스) 추출
Y_target = Iris_Data[['target']]
print(Y_target.sample(10))
```



	petal_len	petal_wd
80	3.8	1.1
140	5.6	2.4
87	4.4	1.3
104	5.8	2.2
107	6.3	1.8
142	5.1	1.9
133	5.1	1.5
59	3.9	1.4
54	4.6	1.5
14	1.2	0.



	target
119	2.0
94	1.0
41	0.0
74	1.0
26	0.0
114	2.0
51	1.0
110	2.0
108	2.0
52	1.0

SVM 모델 GridsearchCV 사용 예

그리드 서치 활용 최적 파라미터 찾기

from sklearn.model_selection import **GridSearchCV**

def svc_param_selection(x, y, nfolds):

```
svm_parameters = [  
    {'kernel': 'rbf',  
     'gamma': [0.1, 0.3, 0.5, 0.7, 1.0],  
     'C': [0.3, 0.7, 1, 1.3, 1.5]}  
]
```

```
clf = GridSearchCV(svm.SVC(), svm_parameters, cv=nfolds) # 10번 교차 검증
```

clf.fit(x, y) # 그리드서치 클래스 생성자에 전달하는 모든 하이퍼파라미터로 교차검증 수행 및 최적 하이퍼파라미터 탐색

이후 최적 하이퍼파라미터로 전체 훈련 세트에 대해 자동으로 다시 모델 훈련

```
print(clf.best_params_)
```

최고 성능일 때의 하이퍼 파라미터

```
print(clf.best_score_)
```

최고 성능일 때의 평가 결과

```
print(clf.best_estimator_)
```

최적 성능을 보여주는 하이퍼파라미터로 학습된 estimator

{'C': 0.3, 'gamma': 0.7, 'kernel': 'rbf'}

0.9666666666666666

SVC(C=0.3, gamma=0.7)

Best_model = clf.best_estimator_ # 최적 estimator 를 저장해 두고 나중에 Test 데이터로 예측 및 정확도 평가

pred = Best_model.predict(X_test)

print('정확도 : ', accuracy_score(X_test, pred))

svc_param_selection(X_petal_len_wd, Y_target.values.ravel(), 10) # 함수 호출

Thank You